DEVELOPER CODEBASE GUIDE

ShopMitra Apna App Samjho

Ye guide aapke apne e-commerce app ka poora naksha samjhati hai — system design, folders, database, login system, order-payment flow, admin panel aur emails. Har chapter aapke hi project ki asli files se examples leta hai, taaki code padhna ek kahani jaisa lage.

ShopMitra Developer Edition

October 2026 · Step-by-step Hinglish learning guide

Vishay Suchi (Table of Contents)

1. ShopMitra: Ek Nazar Me **2**

2. System Design - Bada Naksha **3**

Har manzil ka kaam 3

3. Folder ka Naksha **4**

4. Database ka Design **5**

5. Login aur Password ka System **6**

6. Ek Request ki Journey **7**

7. Seller System - Multi-vendor **8**

8. Order ki Poori Kahani **9**

9. Payment System - Dodo Payments **11**

10. Admin Panel - Aapka Control Room **12**

11. Email System - Resend **12**

12. Ab Aage Kya? - Khud Seekhne ka Plan **13**

1. ShopMitra: Ek Nazar Me

Socho aapne ek badi dukan kholi hai. Is dukan me teen tarah ke log hain: **customer**, jo aake samaan kharidta hai; **seller** (merchant), jo apna maal is dukan me rakhta hai aur bechta hai; aur **super_admin** (aap), jo poore bazaar ka malik hai - kaun beche, kaun na beche, banners kaise dikhein, sab aap tay karte ho. ShopMitra bilkul yahi dukan hai, bas internet par. Ek hi website teen kaam ek saath karti hai: customer ke liye showroom, seller ke liye apna counter, aur aapke liye poore bazaar ka control room.

Technically ye ek **full-stack web application** hai. "Full-stack" ka matlab: jo aap screen par dekhte ho (buttons, cards, colors) aur jo peeche hota hai (data save hona, payment hona, email jaana) - dono cheezein ek hi project me likhi hain. Aapke paas do alag app nahi hain; Next.js naam ka framework dono kaam ek saath karta hai. Neeche wali table me har tool ka naam aur uska kaam hai.

Tool	Kya kaam aata hai	Project me kahan
Next.js 16	Poore app ka framework - pages + server, dono	src/app/
React 19	Screen ke tukde (components) banane ki library	src/components/
TypeScript	JavaScript + galtiyan pehle pakadne wali type safety	*.ts, *.tsx files
Tailwind CSS 4	Styling - class likho, design taiyar	className="..."
Prisma	Database se baat karne ka translator	prisma/schema.prisma
PostgreSQL (Neon)	Database - saara data yahan sota hai	DATABASE_URL (.env)
Dodo Payments	Online payment collect karne ka gateway	src/lib/dodo.ts
Resend	Email bhejne ki service	src/lib/mailer.ts

Table 1: Tech stack - kis cheez ka kya kaam hai

Ab ek zaroori baat samajh lo jisse aadhe concepts apne aap clear ho jayenge: **client aur server ka farak**. Client = user ka browser (phone/laptop). Server = aapka app jahan chal raha hai. Browser sirf "dikhawa" karta hai - buttons, photos, cart ka hisaab. Asli hisaab-kitaab hamesha server par hota hai. Jab aap "Place Order" dabate ho, browser server ko bolta hai "ye order banao"; server khud check karta hai ki stock hai ya nahi, keemat kya hai, aur phir database me order likhta hai. Isi liye koi customer apne browser se total badal kar bhi dhokha nahi de sakta - server uska number dobara khud nikalta hai.

2. System Design - Bada Naksha

Poore system ko ek teen-manzila building ki tarah socho. Neeche ka diagram isi building ka naksha hai: upar browser (jahan user khada hota hai), beech me Next.js server (app ka dimaag), aur neeche data aur services (jahan sab kuch sambhala jaata hai). Ek request upar se neeche jaati hai aur jawab wapas upar aata hai. Is ek picture ko samajh lo - aadhi system design samajh me aa gayi.

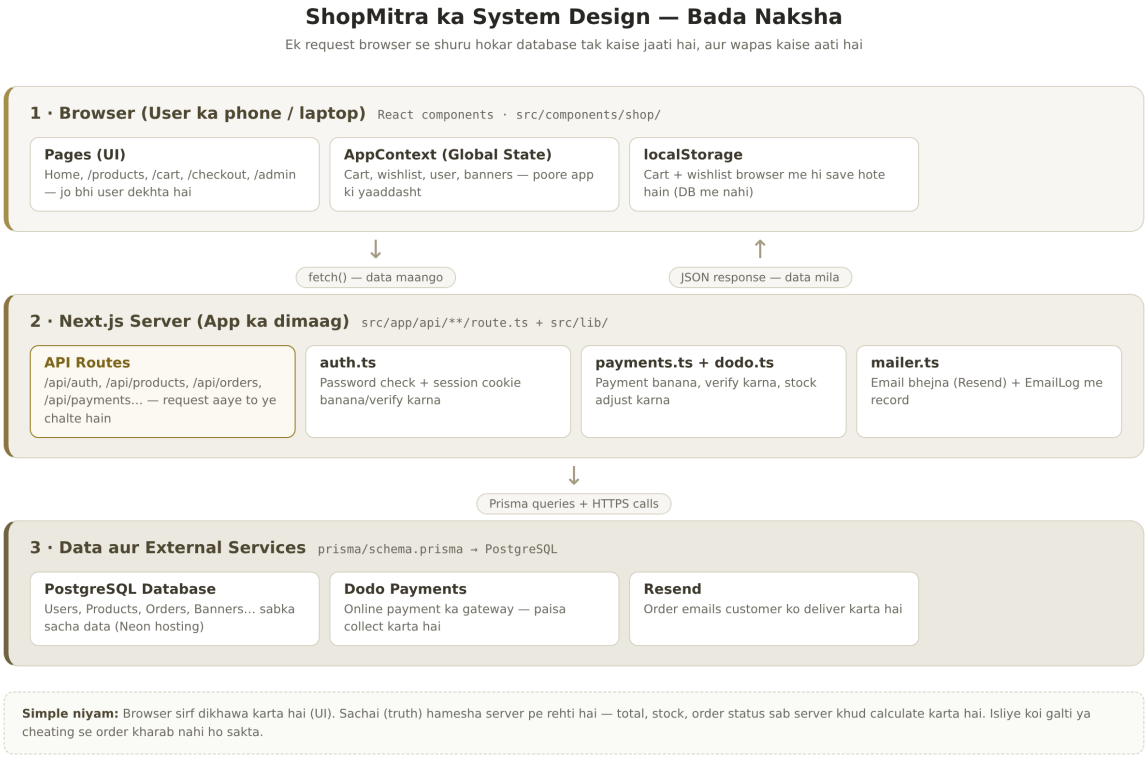


Figure 1: ShopMitra ka system design - teen layers aur unke beech ka raasta

Har manzil ka kaam

Manzil 1 - Browser: Ye user ka phone ya laptop hai. Yahan React ke components chalte hain (Header, ProductCard, Cart page...). Cart aur wishlist is manzil par hi save hote hain - browser ke localStorage me - kyunki wo har user ka apni private cheez hai. Global state (AppContext) isi manzil ka manager hai: poore app ko pata rehta hai cart me kya hai, kaun logged in hai.

Manzil 2 - Next.js Server: Is building ka asli dimaag. Do tarah ke kamre hain: **pages** (src/app ke andar page.tsx files - UI banati hain) aur **API routes** (src/app/api ke andar route.ts files - data ka kaam karti hain). Saath me src/lib folder hai jiske chhote-chhote helpers sab bhaari kaam karte hain: auth.ts (login check), db.ts (database connection), payments.ts + dodo.ts (payment), mailer.ts (email).

Manzil 3 - Data aur Services: PostgreSQL database (Neon par hosted) me saara sacha data soyaa hai - users, products, orders, banners. Dodo Payments paisa collect karta hai, Resend email pahunchata hai. Ye

dono "external services" hain - unka code hamare project me nahi hai, hum unko internet se request bhejte hain jaise kisi dukaan se saman mangwate ho.

> **Sabse zaroori niyam:** Browser jo bhi number bheje, server use dobara khud calculate karta hai. Total, stock, order status - sach hamesha server aur database me rehta hai. Browser sirf request kar sakta hai, sachai nahi.

3. Folder ka Naksha

Ab building ke andar chalte hain - kaun sa kamra kya karta hai. Next.js ke niyam simple hain: **page.tsx** naam ki file = ek web page, **route.ts** naam ki file = ek API (data ka counter), aur [orderNumber] jaisa bracket folder = dynamic page (jis me har order ka apna page banta hai). Neeche wali table aapke project ke har important kamre ka board lagayi hai.

Kamra (file/folder)	Kya hota hai
src/app/page.tsx	Homepage - banners, categories, featured products
src/app/products/, product/[slug]/	Product listing aur detail page
src/app/cart/, checkout/	Cart page aur 3-step checkout
src/app/login/, register/	Login aur naya account (shopper ya seller)
src/app/admin/page.tsx	Seller + super_admin ka poora dashboard (ek hi page, tabs ke saath)
src/app/api/**/route.ts	Saari APIs - auth, products, orders, payments, banners, admin
src/components/shop/	App ke dikhne wale tukde - Header, Footer, ProductCard, PaymentModal...
src/components/ui/	shadcn/ui ke taiyar buttons/inputs (boilerplate)
src/lib/auth.ts	Password hash + session cookie ka pura system
src/lib/db.ts	Prisma database connection (ek hi baar banta hai)
src/lib/dodo.ts	Dodo Payments se baat (checkout banana, payment verify)
src/lib/payments.ts	markOrderPaid / markOrderFailed - payment ka final faisla
src/lib/mailer.ts + emails.ts	Email bhejna + email ke design (HTML templates)
src/lib/serialize.ts	Database ka data browser wale format me badalna
src/lib/utils.ts	Chhote helpers - cart total, order number, price format
src/store/AppContext.tsx	Poore app ki global yaaddasht (cart, user, products...)
prisma/schema.prisma	Database ka blueprint - 11 tables ki definition

Kamra (file/folder)	Kya hota hai
prisma/seed.ts	Shuruaati demo data (categories, products, banners)
scripts/	Dev tools - dev server, db-up, list-users, reset-password...

Table 2: src/ folder - kis file me kya hai

Ek chhota secret: upload/ecomm_extracted folder purana prototype hai (pehle ka version) - usse app ka koi lena-dena nahi. Aur src/components/ui me shadcn ke taiyar-made components pade hain jinme se kuch hi use hote hain. Matlab: asli app src/app + src/components/shop + src/lib + src/store - bas in char kamron me rehta hai. Jab kabhi code dhundhna ho, yahi 4 jagah khojo.

4. Database ka Design

Database ek almirah hai jisme 11 hisaab-kitab ke register (tables) rakhe hain. Har register ki design prisma/schema.prisma naam ki ek hi file me likhi hai - ye file app ka sabse keemti document hai, kyunki yahi batati hai ki data kaisa dikhta hai. Neeche har register ka matlab hai.

Model	Kya store karta hai	Yaad rakhne wali baat
User	Har account - email, passwordHash, name, role, shopName, isActive	role: customer / admin (seller) / super_admin
Product	Har item - naam, price, stock, images, sellerId	sellerId null = platform catalog (seed wala)
Category	Product categories (Electronics, Fashion...)	slug se pehchana jaata hai
Order	Har order - number, total, status, payment status, address	orderNumber unique: SM-XXXXXX-XXXX
OrderItem	Order ke andar ke items (ek order me kai items)	Purchase ke waqt ki price snapshot rakhta hai
Address	User ke saved delivery addresses	User delete to ye bhi delete (Cascade)
Banner	Homepage ke banners	sortOrder se dikhne ka kram
Coupon	Discount codes (SAVE10...) - percentage ya fixed	minOrderAmount check hota hai
Subscriber	Newsletter me judne wale emails	Email unique hota hai
ContactMessage	Contact form se aaye messages	Sirf inbox ki tarah save hota hai
EmailLog	Har email ka record - bheja gaya ya fail hua	Audit ke liye - sab kuch likha rehta hai

Table 3: 11 models - database ke saare registers

Rishtey (relations) kaise judte hain: User ke paas uske orders, addresses aur products hote hain. Order ke andar OrderItems hote hain, aur har OrderItem ek Product se juda hota hai. Delete hone par bhi khayal rakha gaya hai: agar seller ka account delete ho jaye to uske products delete nahi hote - sellerId khaali (null) ho

jaata hai (SetNull); aur agar user delete ho to uske orders bhi bache rehte hain - order history kabhi kho nahi jaati. Sirf Address user ke saath poori tarah juda hai (Cascade) - user gaya to address bhi gaya.

> **Ek technical twist:** Product ki images, tags, specifications aur order ka shippingAddress - ye sab database ke text columns me JSON string ban ke sote hain. serialize.ts unhe browser wale format me kholta hai. Prisma me field ke naam camelCase (orderNumber) hote hain, browser me snake_case (order_number) - ye badalna bhi serialize.ts hi karta hai.

5. Login aur Password ka System

Aapne pucha tha - "sabka password dikha do". Ab samjho ki wo kyun possible nahi hai, aur ye kami nahi, feature hai. Jab bhi koi account banta hai, uska password seedha save NAHI hota. Pehle bcrypt naam ka function use par ek **one-way lock** (hash) laga deta hai. Hash se password wapas padhna namumkin hai - bilkul waise jaise anda se murgha to ban sakta hai, lekin murgha se anda kabhi nahi. Login ke waqt bcrypt aapka likha password aur saved hash ko milata hai aur sirf itna batata hai - "haan, ye mila" ya "nahi mila". Original password kisi ko nahi pata, na aapko, na mujhe, na database admin ko. Isliye bhoolne par "dekho" nahi sakte - sirf "reset" kar sakte ho (naya lock laga do).

```
// src/lib/auth.ts (simplified)
import bcrypt from 'bcryptjs'

export async function hashPassword(password: string) {
  return bcrypt.hash(password, 10)   // 10 rounds ka one-way lock
}

export async function verifyPassword(password: string, hash: string) {
  return bcrypt.compare(password, hash)  // sirf true/false milta hai
}
```

Login hote hi server ek **session token** banata hai - ek chhota sealed certificate jisme likha hai: kaunsa user, kya role, aur kab tak valid (7 din). Is certificate par server ka HMAC signature hota hai - agar koi usme chher-chhad kare, signature match nahi hoga aur request reject ho jayegi. Ye certificate browser me **cookie** ke roop me sambhala jaata hai, aur har baar jab browser koi API call karta hai, cookie apne aap saath chali jaati hai - isse server ko baar-baar password nahi poochna padta.

```
// Har API route ke shuru me yahi hota hai (simplified)
const user = await getSessionUser()   // cookie kholo, signature check
if (!user) {
  return NextResponse.json({ error: "Login required" }, { status: 401 })
}
if (!user.is_active) {
  return NextResponse.json({ error: "Account blocked" }, { status: 403 })
}
```

Property	Value	Matlab
Naam	shopmitra_session	Cookie ki pehchaan
httpOnly	true	JavaScript ise padh nahi sakta - chori mushkil
secure	production me true	Sirf HTTPS par travel karegi
sameSite	lax	Doosri sites isse abuse nahi kar sakti
maxAge	7 din	Uske baad phir se login karna hoga

Table 4: Session cookie ki properties aur unka matlab

Aur haan - block kiya hua account yahin pakda jaata hai. getSessionUser DB se user uthata hai aur isActive check karta hai; band account ka session turant null ho jaata hai, chahe uske browser me cookie padi ho. Block = turant bahar, koi kahani nahi.

6. Ek Request ki Journey

Ab sabse achha tareeka seekhne ka: ek asli request ko shuru se aakhri tak chalte dekho. Lete hain sabse common kaam - products ki list laana. Jab homepage khulta hai, AppContext bootstrap me **/api/products** ko call karta hai. Neeche us route ka simplified version hai - har line ka kaam comment me likha hai.

```
// src/app/api/products/route.ts (simplified)
export async function GET(request: NextRequest) {
  const { searchParams } = new URL(request.url)
  const category = searchParams.get('category')  // ?category=fashion
  const search   = searchParams.get('search')    // ?search=watch

  const products = await db.product.findMany({   // Prisma -> PostgreSQL
    where: {
      isActive: true,        // dikhne wale hi products
      ...(category && { categorySlug: category }),
      ...(search && { name: { contains: search, mode: 'insensitive' } }),
    },
    include: { seller: { select: { shopName: true } } },  // 'Sold by'
  })

  return NextResponse.json(serializeProducts(products))  // JSON jawab
}
```

Yahi pattern har API me hai, bas kaam badalta hai: (1) request aayi, (2) session check - login chahiye ya nahi, (3) input validate (zod schema), (4) Prisma se DB ka kaam, (5) jawab JSON me wapas. Isliye ek route samajh liya to saare routes samajh gaye. HTTP methods ki bhasha bhi yaad rakho - wo batati hai karna kya hai.

Method	Matlab	Example (ShopMitra me)
GET	Data do (sirf padhna)	GET /api/products - sab products
POST	Kuch naya banao	POST /api/orders - naya order
PATCH	Existing cheez update karo	PATCH /api/orders/SM-... - status badlo
DELETE	Hata do	DELETE /api/products/123 - product delete

Table 5: HTTP methods - request ki bhasha

Code	Matlab	Kab milta hai
200	Sab theek	Kaam ho gaya
201	Ban gaya (Created)	Naya order/product bana
400	Galat bheja (Bad Request)	Form me email hi nahi tha
401	Pehchan nahi (Unauthorized)	Login kiye bina protected API maangi
403	Ijazat nahi (Forbidden)	Seller ne doosre ka product badalne ki koshish ki
404	Mil hi nahi (Not Found)	Galat order number

Table 6: Status codes - jawab ka mood

7. Seller System - Multi-vendor

ShopMitra ek dukan nahi, bazaar hai - kai sellers apna-apna maal bechte hain. Ye chalta hai **sellerId** naam ki ek chhoti si cheez se. Jab koi register page par aata hai, usse poocha jaata hai: "Shopper" banna hai ya "Sell on ShopMitra"? Seller chuna to uska role admin ban jaata hai aur shopName bhi leta hai (jaise "Mitea"). Ab se jo bhi product wo banayega, us product par uski stamp lag jaati hai - hamesha ke liye record rehta hai ki ye maal kiska hai.

```
// POST /api/products (simplified) - seller stamping
const admin = await requireAdmin()      // login + role check
if (!admin) return 401                  // bina login ban nahi sakta

const product = await db.product.create({
  data: {
    ...body,                            // naam, price, stock...
    sellerId: admin.id,                 // YE KA MAAL HAI - stamp
    slug: await generateUniqueSlug(body.name),
  },
})
```

Ab **scoping** ka jaadu dekho - "apna hi apna". Jab seller dashboard me products dekhta hai, API sirf uske products laati hai (?manage=1 ke saath). Jab wo kisi doosre seller ke product ko edit ya delete karne ki

koshish karta hai, server check karta hai: product.sellerId === admin.id? Nahi to **403 - "You can only manage your own products"**. Aur super_admin? Uske liye ye lock nahi lagta - wo sab dekh sakta hai, sab badal sakta hai. Customer ke phone par bhi ye seller dikh jaata hai: product page par "Sold by Mitea" jaisa chip lagta hai.

Kaam	Seller (admin)	super_admin
Products dekhna	Sirf apne (manage list)	Sab ke products
Product banana/edit/delete	Sirf apna (warna 403)	Kisi ka bhi
Orders dekhna + status badalna	Jin orders me uska maal ho	Saare orders
Dashboard stats	Apna revenue, apne products	Poore platform + accounts
Users manage / password reset / block	Nahi (403)	Haan, sab kuch
Banners	Admin-level API (dono kar sakte hain)	Haan

Table 7: Seller vs super_admin - kaun kya kar sakta hai

8. Order ki Poori Kahani

Ye is app ka dil hai - customer ke "Place Order" dabane ke baad system me kya-kya hota hai. Neeche ka flowchart ise teen phases me dikhata hai: order banana, payment, aur delivery. Is picture ko dhyan se dekho - phir neeche har phase ki kahani padho.

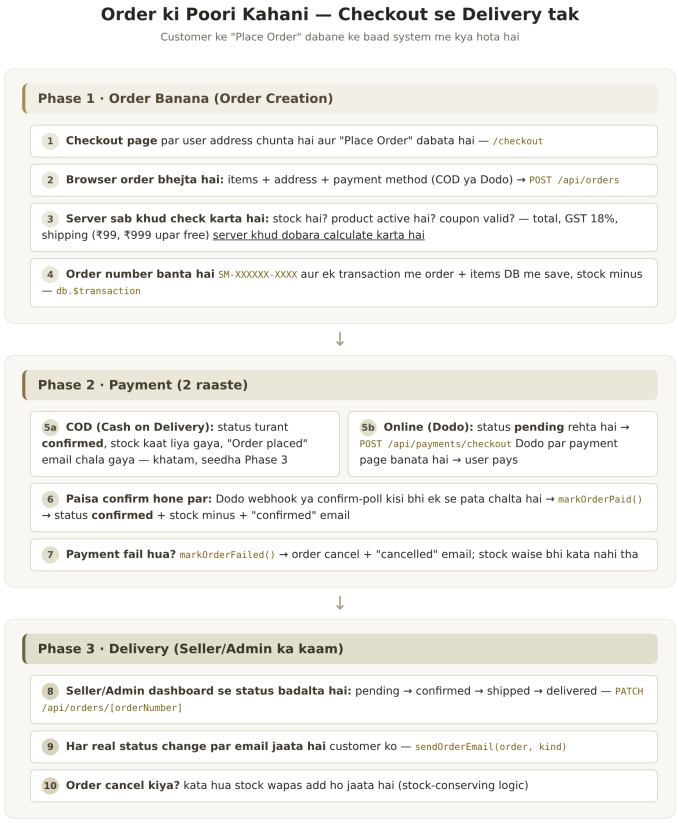


Figure 2: Order lifecycle - checkout se delivery tak teen phases

Phase 1 - Order banana: Checkout page par user address chunta hai aur "Place Order" dabata hai. Browser POST /api/orders par items, address aur payment method (cod ya dodo) bhejta hai. Server ab asli teacher ban jaata hai: har product dobara check karta hai (stock hai? active hai?), keemat DB se uthata hai, coupon validate karta hai, GST 18% aur shipping (Rs 99, Rs 999 se upar free) khud jodta hai. Phir generateOrderNumber() se SM-XXXXXX-XXXX jaisa number banta hai aur ek **transaction** me order + items save hote hain. Transaction ka matlab: ya sab kuch hoga, ya kuch nahi - aadha-adhura order kabhi nahi banta.

Phase 2 - Payment: COD raasta seedha hai - order turant confirmed, stock kaat do, "Order placed" email bhejo. Online raaste me order pending rehta hai jab tak paisa confirm na ho. Paisa confirm hone ke 3 raaste hain (agle chapter me detail) - koi bhi pehle pahunch jaye, markOrderPaid() chal jaata hai: paymentStatus paid, status confirmed, stock minus, aur customer ko confirmed email. Fail hua to markOrderFailed() - cancel + email; stock kata hi nahi tha, to wapas karne ka sawal hi nahi.

Phase 3 - Delivery: Ab seller/admin ka haath: dashboard se status aage badhta hai - confirmed, shipped, delivered. Har real status change par customer ko email jaata hai. Admin order cancel kare to kata hua stock wapas jud jaata hai - isse "stock-conserving logic" kehte hain, taaki cancel hue orders ka maal bechna wapas possible rahe.

> **Kyunki server sab dobara calculate karta hai:** user chahe browser me total 1 rupaya kar de, server apna sahi total hi lega. Ye hi wo design hai jisse app dhokhe aur galtiyon dono se bacha rehta hai.

9. Payment System - Dodo Payments

Online payment ke liye hum Dodo Payments ko beech me rakhte hain - ye ek gateway hai jaise dukaan me card machine. Flow aisa hai: order pending ban chuka hai; ab checkout page POST /api/payments/checkout call karta hai. Server DB se order ka total uthata hai (client ka total kabhi nahi sunta), Dodo par ek checkout session banata hai, aur PaymentModal naam ka overlay user ke saamne khul jaata hai - UPI, card, sab options Dodo dikha deta hai. Dodo ke paas ek poora payment page hai, hum sirf uska window apne page me le aate hain.

Sabse achhi baat: **DODO_PAYMENTS_API_KEY na ho to bhi sab chalta hai**. Tab server khud ek "simulated" checkout banata hai (sim_... wale fake payment ids ke saath) jisse aap dev mode me poora payment flow test kar sakte ho - succeed bhi kar sakte ho, fail bhi. Isliye Dodo account ke bina bhi app kabhi atkta nahi.

Raasta	Kaise kaam karta hai	Kab chalta hai
Confirm poll	PaymentModal har 2.5 second me POST /api/payments/confirm poochta hai	Jab tak payment window khula hai
Return page	Dodo user ko /payment-return par wapas bhejta hai; page confirm API ko final status batata hai	Payment ke turant baad
Webhook	Dodo ka server seedha hamare /api/payments/webhook par khabar bhejta hai (signature ke saath)	Safety net - browser band bhi ho to

Table 8: Payment confirm hone ke 3 raaste

Webhook sabse interesting hai: Dodo ka server khud hamare server ko bolta hai "payment.succeeded hua hai" - ye ek server-to-server khabar hai jise verifyDodoWebhook() signature se verify karta hai (taaki koi jhooti khabar na bhej sake). Teeno raste ek hi jagah milte hain: markOrderPaid(). Ye function **idempotent** hai - matlab 100 baar bhi chale to order sirf ek baar paid hoga, stock sirf ek baar katega. Double-payment ka koi bhay nahi.

10. Admin Panel - Aapka Control Room

/admin page aapka control room hai - ek hi page par 6 tabs: Dashboard, Products, Orders, Banners, Customers aur Settings. Seller isme apni dukan dekhta hai; aap (super_admin) poora bazaar. Har button ke peeche ek API hai - neeche wali table me "kya karna hai" aur "kaun sa kaam karega" dono diye hain. Jab bhi panel me koi naya feature chahiye, pehle socho kaunsa API chalega - wahi pattern har jagah hai.

Kya karna hai	Kahan jao (UI)	Peeche ka API
Kitne users/merchants hain	Dashboard - Accounts strip	GET /api/admin/stats
Kisi ko block/unblock	Customers tab - Block button	PATCH /api/admin/users/[id]
Password set karna	Customers tab - Reset password	PATCH /api/admin/users/[id] (password)
Role badalna (seller banana)	Customers tab - role dropdown	PATCH /api/admin/users/[id] (role)
Banner badalna	Banners tab - Edit	PATCH /api/banners/[id]
Order ka status badalna	Orders tab	PATCH /api/orders/[orderNumber]
Product hatana/edit	Products tab	PATCH / DELETE /api/products/[id]

Table 9: Control room - kaam aur unke APIs

Password wala button par ek baar phir gaur karo: panel password "dikhata" nahi, "naya set" karta hai - kyunki hash wapas padhna namumkin hai. Reset karte hi bcrypt naya hash bana kar DB me rakh deta hai, purana lock khatam. Aur ek safety: aap khud ko block ya demote nahi kar sakte - API khud ko 400 bhej deta hai (self-lockout protection), taaki galti se poora platform kabhi bina owner na reh jaye.

11. Email System - Resend

Har bade moment par customer ko email jaata hai - order laga, paisa mila, saman chala, pahunch gaya, cancel hua. Ye poora system do files me hai: **emails.ts** email ka design banata hai (HTML template - brand colors, order ki table, track-order link) aur **mailer.ts** use Resend ko bhejta hai. Chain aisi hai: orders API ya payments.ts bolta hai sendOrderEmail(order, kind) - wo template render karta hai - sendMail Resend ko deta hai - aur har koshish (bheji ya fail) EmailLog table me likhi jaati hai. Audit kabhi bhi kar sakte ho: kaunsa email kisko gaya, kab, kya status.

Email (kind)	Kab jaata hai	Kise se
placed	COD order ban gaya	orders API (POST)
confirmed	Payment success hua	markOrderPaid()

Email (kind)	Kab jaata hai	Kise se
shipped	Admin ne shipped kiya	orders PATCH
delivered	Order pahunch gaya	orders PATCH
cancelled	Cancel hua (payment fail ya admin cancel)	markOrderFailed() / PATCH

Table 10: Paanch email - kab kaun jaata hai

Ek zaroori safety design: emails **fire-and-forget** hain - code "void sendOrderEmail(...)" likhta hai aur aage badh jaata hai, email ke complete hone ka wait nahi karta. Matlab agar Resend down ho jaaye, customer ka order phir bhi turant banega - email thodi der baad ya kabhi nahi bhi jaye to business nahi rukta. Aur RESEND_API_KEY na ho to dev mode me email console par chhap jaata hai - testing ke liye perfect. Guest orders (bina login) par email skip hota hai - kisko bhejenge, address hi nahi.

12. Ab Aage Kya? - Khud Seekhne ka Plan

Ab aapke paas naksha hai. Seekhne ka akela tareeka hai: haath gande karo. Pehle ye commands yaad kar lo - ye aapke roz ke hathiyaar hain (sab project folder se chalao, jo pehle se bun/npm dono chalate hain).

Command	Kya karta hai
bun run dev	Dev server chalao - localhost:3000 par app khulega
bun run db:push	schema.prisma ka badla hua design database me apply karo
bun run db:studio	Database ka visual editor kholo (tables dekho, data dekho)
bun run db:seed	Shuruaati demo data dobara bharo (categories, products, banners)
node scripts/list-users.mjs	Saare accounts ki list (email, role, shop)
node scripts/list-product-owners.mjs	Kaun sa product kiska hai
node scripts/reset-owner-password.mjs email newpass	Kisi ka password reset (DB se)
bun run lint	Code ki galtiyan check karo

Table 11: Roz ke commands

Ab paanch exercises - har ek 5-10 minute leti hai aur har ek aapko ek naya kamra dikhayegi. Inhe karte waqt sirf dekhna nahi, karke dekho:

1. **Banner banao:** /admin kholo, Banners tab, naya banner add karo - homepage par turant dikhega. Peeche POST /api/banners chala hoga.
2. **Apna product becho:** apne account (super_admin) se Products tab me naya product add karo, phir uska detail page kholo - "Sold by" chip dekho.

3. **Customer ban ke kharido:** nayi incognito window me shopper account banao, 2 items cart me dalo, COD se order karo. Phir admin window me us order ka status badlo aur dekho customer ko email (EmailLog me record).
4. **Block ka khel:** Customers tab me us customer ko Block karo. Ab uske browser me kuch bhi karo - session mar chuka hai. Unblock karke wapas lao.
5. **Code me jhanko:** src/app/api/orders/route.ts kholo aur sirf padho - upar wale chapters ki har cheez wahan milegi: session check, zod, transaction, email. Jo samajh na aaye, wo isi guide ke chapter se milao.

Shabd	Matlab (ek line me)
API	Do programs ki baat-cheet ka raasta - "mujhe ye data do"
Route	Ek URL ka address - /api/products ek route hai
Schema	Database ke tables ka blueprint (schema.prisma)
Migration / db:push	Schema ke badlaav ko database tak pahunchana
Seed	Shuruaati demo data bharna
Hash	One-way lock - password ko padhna-mushkil banake rakhna
Cookie	Browser me rakhi chhoti tikiya (session certificate)
Session	Server ka yaad rakhna - "ye user logged in hai"
Webhook	Doosri service ka humein khud khabar dena (Dodo se)
Transaction	Ya sab kaam hoga, ya kuch nahi - aadha kaam kabhi nahi
Env variable	.env file ke secrets (API keys, database URL)
Idempotent	Baar-baar chalao, natija ek hi baar jaisa

Table 12: Glossary - naye shabd

Aakhri baat: code padhna kahani padhne jaisa hai. Har route.ts ek chhoti kahani hai - shuruat session check se, beech me database ka kaam, ant me jawab. Pehle sirf padho, phir chhote badlaav karo, phir naya feature likho. Tin mahine me aap khud ko code likhte hue paaoge. Aur jab kabhi atak jao, mujhse poochna - ye guide isi liye to hai.